

Hardware Trojan Detection with Machine Learning and Power Side-Channels: A Post-Deployment Analysis

Ashwin Koshy John¹, Sai Tarrun Pitta², Jaya Dofe¹ and Jai Gopal Pandey³

¹Department of Electrical and Computer Engineering, California State University, Fullerton, CA, USA

²Department of Computer Science, California State University, Fullerton, CA, USA

³CSIR- Central Electronics Engineering Research Institute (CEERI), Pilani- 333031, India

Email ID: jdofe@fullerton.edu

Abstract—The detection of hardware Trojans has become increasingly complex due to evolving attack vectors and the growing separation between IP owners and foundries, a trend driven by the industry's shift toward fabless manufacturing. Hardware Trojans involve malicious modifications to hardware designs that create exploitable vulnerabilities. Traditional detection techniques, such as reverse engineering and side-channel analysis, have proven effective but often come with high costs and substantial preprocessing overhead. This paper explores machine learning algorithms combined with side-channel analysis to detect dormant hardware Trojans. We collected power consumption data from an Artix FPGA running an AES-128 algorithm that included various Trojan insertions. Experimental results show that applying machine learning directly to power consumption traces allows for effective Trojan detection without extensive feature extraction. Among the classifiers evaluated, the Random Forest model achieved a detection accuracy of 98%, outperforming existing methods. The K-Nearest Neighbors (KNN) classifier also showed strong performance, achieving an accuracy of 97%. Our proposed method reduces preprocessing requirements, lowers computational costs, and improves noise robustness, thereby enabling the post-deployment detection of Trojans in integrated circuits.

Index Terms—Hardware Trojans, Hardware Trojan detection, Machine learning, Side-channel analysis, Supervised learning

I. INTRODUCTION

The industry's shift toward fabless manufacturing has intensified the need for strong hardware security, as trust between IP owners and foundries has become increasingly critical [1], [2]. This reliance on third-party foundries introduces vulnerabilities, particularly the risk of hardware Trojans being implanted during manufacturing. These Trojans are often small, activated under rare conditions, and difficult to detect. The increasing sophistication of hardware Trojan attacks further complicates detection efforts. Common detection methods include reverse engineering, logic testing, and side-channel analysis. Previous research has proposed using reverse engineering to identify tampered integrated circuits (ICs) [3], [4]. Although reverse engineering is considered the most reliable method for detecting hardware Trojans, it is a destructive and invasive process that demands substantial time, manual effort, and resources [4], [5]. Techniques like K-means clustering, backscattering side-channel clustering etc. are used to improve the efficiency of

reverse engineering tests [6]. While effective, these approaches are highly time-consuming [7]. Logic testing identifies hardware Trojans by analyzing a circuit's output under specific test patterns. However, selecting effective test patterns is challenging for large circuits [8]. This method also has limited coverage and often misses Trojans that are strategically inserted and rarely triggered. Trojans that don't produce noticeable output changes due to a lack of functional payload remain undetectable by this method.

Side-channel testing is a widely used technique for detecting hardware Trojans by analyzing variations in side-channel data [6], [7]. This approach measures side-channel parameters and compares them to expected baseline values, where deviations may indicate covert modifications or malicious insertions. Common side-channels include backscattering, power consumption, leakage current, temperature, and electromagnetic emissions, or a combination of these factors [9]. Unlike reverse engineering, side-channel detection is non-invasive, as it analyzes external signals without modifying the IC. This enables testing on a larger population of chips without damaging the hardware, making it a scalable and efficient detection method. In addition to traditional detection methods, machine learning algorithms are crucial for hardware Trojan identification, leveraging advanced techniques in feature extraction and pattern recognition [6], [10]–[13]. These methods are vital for accurately identifying and characterizing subtle deviations in side-channel data that may indicate the presence of hardware Trojans. Recent advances in hardware security highlight the effectiveness of machine learning in detecting these deviations [14]. **Particularly, Machine learning-based run-time monitoring offers a robust solution for identifying Trojans that might bypass initial testing, enhancing the ability to detect such threats in real time [15], [16].**

Many existing studies apply machine learning techniques for hardware Trojan detection, often with a strong reliance on extensive preprocessing and/or explicit Trojan activation [7], [13], [17]–[20]. **This work addresses an important gap by exploring machine learning-based runtime detection for small, inactive Trojans. Our approach utilizes non-invasive side-channel data without modifying the Device Under Test**

(DUT), enabling Trojan detection during normal operation without disrupting functionality. The main objective of this paper is to demonstrate how commonly used machine learning techniques can significantly enhance detection accuracy for subtle and hard-to-detect Trojans in deployed ICs. Our contributions are threefold. First, we improve the efficiency of hardware Trojan detection for lightweight applications by eliminating the need for extensive preprocessing. Specifically, we leverage raw power traces as direct input to machine learning models, avoiding potential biases of manual feature extraction while preserving subtle signal variations critical for Trojan detection. Second, we validate our approach by testing a diverse set of hardware Trojans sourced from existing literature. Finally, we compare the detection accuracy and execution time of commonly used machine learning algorithms applied to the same hardware Trojans, allowing us to identify the most effective detection method for given application.

II. PRELIMINARIES

A. Vulnerable IC Manufacturing Flow

Figure 1 illustrates the manufacturing flow of modern IC production [2]. Hardware Trojans can be introduced at various stages of this process, which is critical to consider when evaluating detection methods. Detection at the Register Transfer Level stage is insufficient, as it overlooks Trojan insertions that may occur after the design phase or during fabrication. In fact, hardware Trojan insertion at the foundry is the most common scenario, as many IC companies rely on offshore foundries for chip fabrication, which are inherently more challenging to safeguard. Therefore, it is critical to develop detection techniques that cover the entire lifecycle of the IC, including post-fabrication stages.

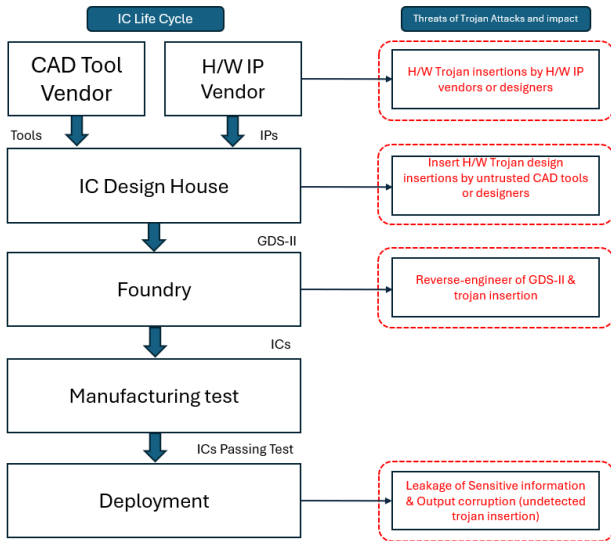


Fig. 1: Overview of Modern IC Manufacturing Process Highlighting Examples of Hardware Trojan Insertion Points

B. Hardware Trojan

A hardware Trojan is a malicious modification introduced by an attacker to alter the functionality of an integrated circuit (IC) [2], [21]. Hardware Trojans can be categorized based on their activation mechanisms, physical properties, and payload characteristics. These insertions pose significant risks to system functionality, security, and reliability, particularly in critical applications such as medical and military equipment. Consequently, they have emerged as a major concern within the semiconductor industry.

C. Machine Learning Algorithms

Machine learning enhances hardware Trojan detection by analyzing large datasets and revealing patterns often missed by traditional methods. This study addresses the challenge of detecting dormant hardware Trojans using machine learning models on raw power traces. To tackle this, we selected a diverse set of classifiers—Random Forest, K-Nearest Neighbors (KNN), and Convolutional Neural Networks (CNN) to capture different characteristics of side-channel data. This approach facilitates robust Trojan detection with minimal preprocessing, contributing to a growing body of work focused on lightweight and scalable Trojan detection solutions.

1) *Random Forest*: Random Forest is a non-parametric machine learning algorithm that combines multiple decision trees to reduce overfitting and bias. Its robustness makes it effective for hardware Trojan detection using power analysis, as it can handle complex data and subtle variations in power consumption. In this study, hyperparameter tuning is performed using grid search over the number of estimators and tree depth to optimize model performance.

2) *KNN*: KNN is a non-parametric algorithm that classifies data based on similarity in feature space. While effective with large datasets, it is computationally intensive and prone to overfitting in high dimensions. In this study, hyperparameter tuning is performed using grid search over the number of neighbors and weighting schemes to optimize performance on power trace data. Despite its simplicity, KNN demonstrated strong pattern recognition capabilities, making it a valuable model for hardware Trojan detection via side-channel analysis.

3) *CNNs*: CNNs are widely used deep learning models known for their effectiveness in extracting hierarchical features from sequential and spatial data. In this work, a lightweight CNN is implemented to learn patterns from power traces, enabling hardware Trojan detection with a balance of performance and model simplicity.

III. RELATED WORK

S. Sun et al. [22] used EM radiation combined with transfer learning for Trojan detection in AES. However, the method's dependence on EM probe resolution can obscure Trojan-induced differences. Rice et al. [23] explored machine learning for Trojan detection in AES-128 FPGA, achieving 97% accuracy but without considering practical Trojan insertions that may produce more complex power signatures. The work

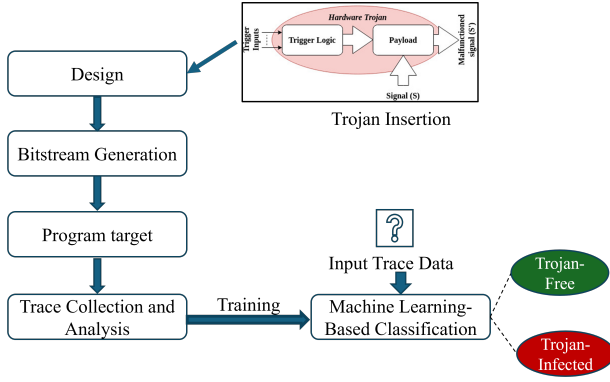


Fig. 2: Methodology Workflow

in [24] introduced a one-class SVM with majority voting for hardware Trojan detection, improving accuracy and reducing golden model dependency. However, the majority voting step increases processing complexity, which may limit scalability and real-time applicability. A recent study [20] applied support vector machines, neural networks, and decision trees with extensive feature extraction from time, frequency, and wavelet domains of power side-channel data; similarly, the HT-EMIS framework [13] proposed a runtime EM side-channel analysis workflow using convolutional neural networks, and another work [25] utilized continuous wavelet transforms combined with an improved ConvNeXt network for detection. Although these methods achieved high detection accuracy, they share limitations such as significant computational resource requirements, sensitivity to noise, and reliance on fixed preprocessing parameters, which may impact robustness in practical applications.

IV. PROPOSED METHODOLOGY AND EXPERIMENTAL SETUP

We present our methodology for detecting hardware Trojans using power side-channel analysis and machine learning.

A. Proposed Approach Overview

Figure 2 illustrates the detailed workflow of our proposed detection approach. Clean and Trojan-inserted hardware designs are synthesized and programmed onto the target device, from which power traces are collected during operation. Unlike traditional methods requiring extensive preprocessing, our approach leverages raw traces, simplifying the detection process. These traces are used to train supervised and unsupervised machine learning models to classify designs as Trojan-free or Trojan-infected.

B. Hardware Setup

Power traces for the experiment are collected using the CW1200: ChipWhisperer-Pro [26]. The ChipWhisperer hardware employs a synchronous capture methodology on the DUT, which significantly reduces the required sample rate [27]. The CW1200 can sample up to 105 MS/s with a large sample

buffer capacity of 98k samples [26]. The advanced hardware trigger modules shown in Figure 3, including IO (UART and SPI) allow for flexible triggering during data capture. The experiment use triggering through the IO, the FPGA would set the trigger pin (ti04) high during encryption. These advanced triggers enhance the ability to pinpoint specific events during power analysis. Experimental analysis is carried out using the ChipWhisperer CW305 Artix FPGA Target, as shown in Figure 4. This target board facilitates the execution of both Trojan-free and Trojan-infected AES-128 implementations on the Artix FPGA. A custom USB interface enables bitstream flashing, data transmission, and adjustment of the target PLL operating frequencies. Power traces are collected through a +20 dB low-noise amplifier integrated into the target board. Each capture consists of 129 samples, with a total of 10,000 traces collected per bitstream for side-channel analysis using machine learning.

C. Trojan Modeling

Table I, shows various Trojan implementations used to evaluate machine learning techniques in hardware Trojan detection. Integrated into AES-128, side-channel power data is analyzed with four Trojan-infected bitstream variations, representing common Trojan types from the literature.

- Trojan T1: This Trojan is modeled based on TrustHub benchmark circuit AES-T2500 [28] and incorporates a 4-bit synchronous counter into the base circuit. The Trojan is activated when the fourth bit of the counter is high, causing it to corrupt the encryption output by flipping the least significant bit of the ciphertext.
- Trojan T2: This trojan is also based on TrustHub benchmark circuit AES-T2300 [28]. It is triggered when two rarely activated nodes within the original design are high. When activated, the Trojan inverts the least significant bit of the output.
- Trojan T3: Based on the Trojan design presented in [29], this Trojan is triggered when a specific key is input into the system. This insertion simulates a Denial of Service (DoS) attack as the encrypted cipher text can only be decrypted by an attacker possessing the predefined key.
- Trojan T4: This Trojan inserts an 11-stage ring oscillator, activated by a subnode in the original design. Implemented as LUTs forming a combinational loop on FPGA, it mimics the insertion of ring oscillators, which can degrade performance, cause failures, or increase power consumption without affecting normal operation [30], [31].

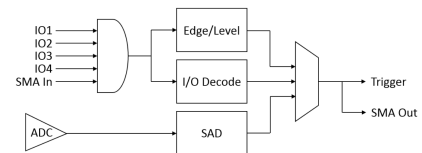


Fig. 3: Trigger Module

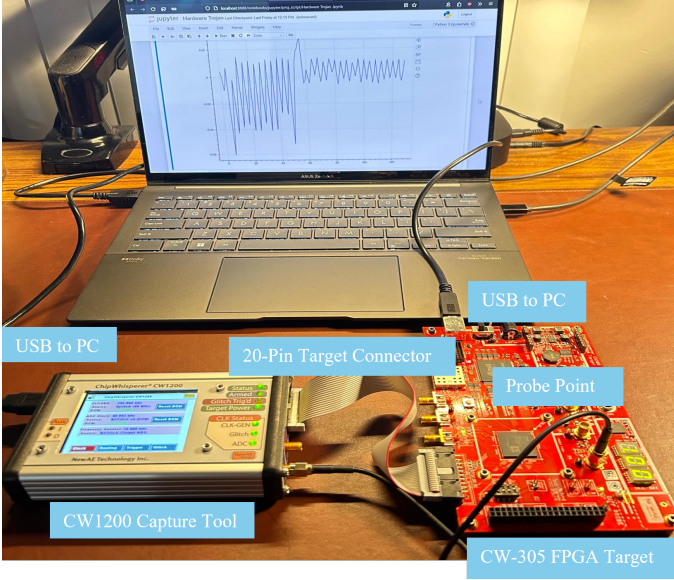


Fig. 4: Experimental Setup for Collecting Side-channel Data

Table I compares the resource utilization ratios of Trojan-inserted designs to the original circuit, assessing how different Trojan types and sizes impact resource usage. This comparison helps evaluate the effectiveness of the proposed detection approach, especially for Trojans with minimal utilization. Power traces are collected while the Trojans are inactive, simulating real-world conditions where adversaries design Trojans with rare activation triggers to evade detection during functional testing.

V. EXPERIMENTAL RESULTS

A. Power Analysis and Trojan Impact

We first analyze the collected power traces to assess the effectiveness of the Trojan models and ensure they do not introduce excessive deviations in peak power consumption. To better utilize side-channel power data for classification, we opt not to rely on traditional features like minimum, maximum, average, and peak power. Instead, we employ advanced algorithms such as Random Forest, KNN, and CNN, which handle data non-linearity, noise robustness, and high dimensionality more effectively. These algorithms are trained to detect Trojan insertions while minimizing the impact of process variations. We compare peak power consumption data to identify discrepancies between the two categories, finding no substantial differences, except for the T2 Trojan. Figure 5 illustrates the peak power utilization of Trojan-inserted designs



Fig. 5: Peak Power Characteristics in Trojan free and Infected Traces

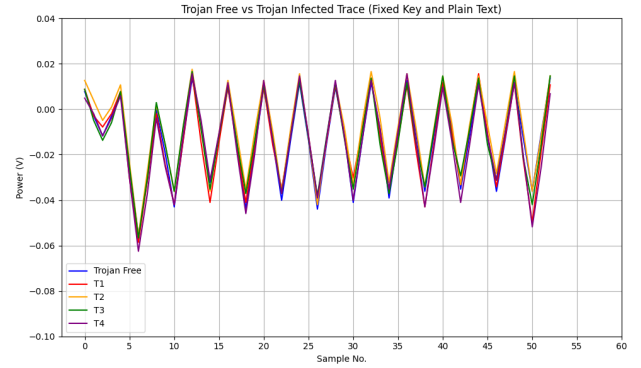


Fig. 6: A single Trace for Trojan-free and Infected Encryption with Fixed Key and Variable Input

compared to normal implementations, highlighting the subtlety of Trojan effects.

We collect a batch of traces using the same plaintext and key to verify power consumption. Each trace contains 55 samples, as the encryption process completes in fewer than 60 samples with a $4\times$ ADC clock provided by the FPGA, which synchronizes the capture hardware. Figure 6 demonstrates that individual traces exhibit minimal variation in their

Trojan Characteristics				Trojan Size
Trojan	Type	Trigger	Payload	Percentage of Trojan Utilization
T1	Sequential	Asynchronous counter	Incorrect Output	0.1%
T2	Combinational	Rare Nodes	Incorrect Output	0.05%
T3	Combinational	Input Key	Attacker specified Key	0.03%
T4	Ring Oscillator	Rare Node	Combinatorial Loop	7.3%

TABLE I: Trojan Characteristics and Utilization Comparison

TABLE II: Confusion Matrix and Performance Metrics of Random Forest, KNN, and CNN Models

Model	TN	FP	FN	TP	Accuracy	Precision	Recall	F1-Score
Random Forest	1952	48	15	7985	0.9937	0.9940	0.9981	0.9961
KNN	1803	197	29	7971	0.9774	0.9759	0.9964	0.9860
CNN	1891	109	95	7905	0.9796	0.9864	0.9881	0.9873

power consumption profiles. The peak power measurements for triggered Trojans T1, T2, T3, and T4 are 0.037 mW, 0.039 mW, 0.035 mW, and 0.041 mW, respectively—values that are close to the baseline AES peak power of 0.035 mW. Such small deviations may easily be masked by process variations, potentially allowing hardware Trojans to evade detection.

We acquire power traces for each Trojan variant and for the baseline AES implementation. To enhance signal quality and reduce noise, we enable *trace averaging in the Chip-Whisperer GUI, capturing multiple traces per measurement and averaging them before saving*. The resulting traces are labeled as either “Trojan-Free” or “Infected” for training the machine learning models. We perform 10-fold cross-validation to ensure a balanced and comprehensive evaluation of model performance, minimizing the risk of overfitting and avoiding skewed accuracy metrics due to class imbalance [32]. Each trace is collected using a fixed key and variable plaintext to reduce variability introduced by key differences, thereby focusing on the chip’s behavior and ensuring measurement consistency.

Initial analysis shows the data is non-linearly separable, as highlighted in prior work [33]. This study therefore focuses on machine learning algorithms that capture complex, non-linear relationships. The performance metrics in Table II reflect model behavior across different Trojan insertions. Among them, *recall* is especially important, as it measures the models’ ability to correctly identify Trojan-infected instances [34].

B. Experimental Observations and Discussion

The performance of three machine learning models: Random Forest, KNN and CNN is assessed using standard classification metrics, including accuracy, precision, recall, and F1-score. In this comparative analysis, Random Forest emerges as the most effective model, achieving an accuracy of 99%, a precision of 99%, and a recall of 99%. These metrics indicate a robust classification capability and minimal misclassification rates, making Random Forest superior for hardware Trojan detection. KNN achieves an accuracy of 97.7%, precision of 97.6%, and recall of 99.6%, reflecting strong classification performance. CNN follows a similar trend, with 97.9% accuracy, 98.6% precision, and 98.8% recall. However, both models fall short of Random Forest, which achieves the highest accuracy, precision, and recall. The higher false positive and false negative rates in KNN (197 and 29) and CNN (109 and 95) suggest they are slightly less effective at consistently distinguishing between clean and Trojan-inserted chips, with CNN showing better precision but a larger number of missed Trojans compared to KNN.

While accuracy is crucial, computational cost cannot be overlooked. Random Forest provides the highest accuracy but

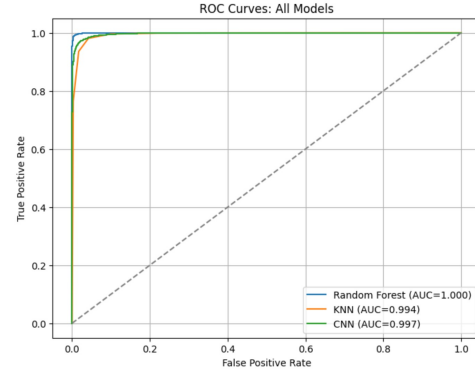


Fig. 7: ROC Curve Comparing Classification Performance of Different Models

requires 100% relative training time, making it slower than KNN, which trains in only 24% of that time and is the fastest model. Although both models achieve similar accuracy, CNN demands the most training resources (400%). This highlights the trade-off in choosing a model for hardware Trojan detection: it requires balancing accuracy with training time and resource limitations.

In addition to the performance metrics discussed above, the area under the ROC curve (AUC), shown in Figure 7, measures each model’s ability to distinguish between classes, with values closer to 1 indicating better performance. The Random Forest model achieved an AUC of 1.00, while the KNN and CNN models reached AUCs of 0.99 and 0.90, respectively, consistent with their precision, accuracy, and F1 scores reported earlier.

We compare the proposed approach with existing methods reported in the literature. As shown in Table III, the Random Forest model achieves 99% accuracy, slightly surpassing the previously reported 97%. Our KNN reaches 98%, outperforming earlier 50% and 81.5% results. Our optimized CNN model achieves 98% accuracy, approaching the 100% benchmark reported in [13]. The difference in CNN performance may stem from their use of EM side-channel data, which offers higher spatial resolution for Trojan activation detection and a deeper CNN architecture with residual blocks, compared to our simpler model trained on raw power trace data.

TABLE III: Performance Comparison of Proposed Models with Existing Literature

Model	Accuracy (%)		
	[20]	[13]	Proposed
Random Forest	97	N/A	99
KNN	50	81.5	98
CNN	N/A	100	98

VI. CONCLUSION

The proliferation of fabless manufacturing has increased the risk of hardware Trojan attacks, necessitating advanced detection strategies. This study explores the detection of dormant hardware Trojans in an AES-128 FPGA implementation using raw power side-channel data combined with supervised machine learning. Among the models tested, Random Forest achieved the highest accuracy (99%), followed by CNN (97.74%) and KNN (97.74%). The results suggest tree-based methods like Random Forest handle power trace noise effectively, while CNN demonstrates competitive performance with an efficient architecture.

While the supervised learning approach produces strong results, its real-world applicability is limited by the difficulty of obtaining labeled Trojan-infected and golden chips at scale. In future work, we plan to explore unsupervised and semi-supervised techniques that do not rely on labeled Trojan-infected data, focusing instead on golden chip data to support practical deployment in real-world supply chains.

REFERENCES

- [1] M. Xue, "Ten years of hardware trojans: a survey from the attacker's perspective," *IET Computers Digital Techniques*, vol. 14, pp. 231–246(15), November 2020.
- [2] S. Bhunia, M. S. Hsiao, M. Banga, and S. Narasimhan, "Hardware Trojan Attacks: Threat Analysis and Countermeasures," *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1229–1247, 2014.
- [3] S. Rajendran and M. L. Regeena, "A Novel Algorithm for Hardware Trojan Detection Through Reverse Engineering," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 4, pp. 1154–1166, 2022.
- [4] C. Bao, D. Forte, and A. Srivastava, "On Reverse Engineering-Based Hardware Trojan Detection," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 1, pp. 49–57, 2016.
- [5] J. Dofe, W. Danesh, V. More, and A. Chaudhari, "Natural language processing for hardware security: Case of hardware trojan detection in fpgas," *Cryptography*, vol. 8, no. 3, 2024. [Online]. Available: <https://www.mdpi.com/2410-387X/8/3/36>
- [6] L. N. Nguyen, C.-L. Cheng, M. Prvulovic, and A. Zajić, "Creating a Backscattering Side Channel to Enable Detection of Dormant Hardware Trojans," *IEEE Transactions on Very Large Scale Integration Systems*, vol. 27, no. 7, pp. 1561–1574, 2019.
- [7] C. He, D. Lei, H. Wu, L. Cheng, G. Yan, and Q. Huang, "A Side-Channel Hardware Trojan Detection Method Based on Fuzzy C-Means Clustering and Fusion Distance Algorithms," *IEEE Internet of Things Journal*, vol. 11, no. 8, pp. 13927–13937, 2024.
- [8] Z. Pan and P. Mishra, "Automated Test Generation for Hardware Trojan Detection using Reinforcement Learning," in *2021 26th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2021, pp. 408–413.
- [9] A. N. Nowroz, K. Hu, F. Koushanfar, and S. Reda, "Novel Techniques for High-Sensitivity Hardware Trojan Detection Using Thermal and Power Maps," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 12, pp. 1792–1805, 2014.
- [10] S. Faezi, R. Yasaei, and M. A. Al Faruque, "Htnet: Transfer learning for golden chip-free hardware trojan detection," in *2021 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2021, pp. 1484–1489.
- [11] Z. Pan and P. Mishra, "TD-Zero: Automatic Golden-Free Hardware Trojan Detection Using Zero-Shot Learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 7, pp. 1998–2011, 2024.
- [12] S. Faezi, R. Yasaei, A. Barua, and M. A. A. Faruque, "Brain-Inspired Golden Chip Free Hardware Trojan Detection," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2697–2708, 2021.
- [13] H. Wang et al., "HT-EMIS: A Deep Learning Tool for Hardware Trojan Detection and Identification through Runtime EM Side-Channels," in *Proceedings of the Great Lakes Symposium on VLSI*, 2023, p. 51–56.
- [14] M. Tehranipoor, "New Directions in Hardware Security," in *2016 29th International Conference on VLSI Design and 2016 15th International Conference on Embedded Systems (VLSID)*, 2016, pp. 50–52.
- [15] Y. Liu, G. Volanis, K. Huang, and Y. Makris, "Concurrent hardware Trojan detection in wireless cryptographic ICs," in *2015 IEEE International Test Conference (ITC)*, 2015, pp. 1–8.
- [16] S. Rajput, J. Dofe, and W. Danesh, "Automating hardware trojan detection using unsupervised learning: A case study of fpga," in *2023 24th International Symposium on Quality Electronic Design (ISQED)*, 2023, pp. 1–6.
- [17] L. N. Nguyen, B. B. Yilmaz, M. Prvulovic, and A. Zajic, "A Novel Golden-Chip-Free Clustering Technique Using Backscattering Side Channel for Hardware Trojan Detection," in *2020 IEEE International Symposium on Hardware Oriented Security and Trust*, 2020, pp. 1–12.
- [18] J. He, Y. Zhao, X. Guo, and Y. Jin, "Hardware Trojan Detection Through Chip-Free Electromagnetic Side-Channel Statistical Analysis," *IEEE Transactions on Very Large Scale Integration Systems*, vol. 25, no. 10, pp. 2939–2948, 2017.
- [19] A. Ghimire, F. Amsaad, T. Hoque, K. Hopkinson, and M. T. Rahman, "Unsupervised ic security with machine learning for trojan detection," in *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*, 2023, pp. 20–24.
- [20] N. Bhatta and F. Amsaad, "ML assisted techniques in power side channel analysis for trojan classification," *Cluster Computing*, vol. 28, 01 2025.
- [21] Z. Huang, Q. Wang, Y. Chen, and X. Jiang, "A Survey on Machine Learning Against Hardware Trojan Attacks: Recent Advances and Challenges," *IEEE Access*, vol. 8, pp. 10796–10826, 2020.
- [22] S. Sun, H. Zhang, X. Cui, L. Dong, and X. Fang, "Electromagnetic Side-Channel Hardware Trojan Detection Based on Transfer Learning," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 3, pp. 1742–1746, 2022.
- [23] V.-P. Hoang, "Hardware Trojan detection based on side-channel analysis using power traces and machine learning," *Target*, vol. 2, p. 0, 2021.
- [24] D. Jap, W. He, and S. Bhasin, "Supervised and unsupervised machine learning for side-channel based trojan detection," in *2016 IEEE 27th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2016, pp. 17–24.
- [25] Y. Gao, J. Su, J. Li, S. Wang, and C. Li, "A neural network framework based on convnet for side-channel hardware trojan detection," *ETRI Journal*, vol. 47, pp. 338–349, 04 2024.
- [26] "Cw1200: Chipwhisperer-pro," 2021, accessed: 2024-09-06. [Online]. Available: https://www.mouser.com/datasheet/2/894/NAE-CW1200_datasheet-1877401.pdf
- [27] C. O'Flynn and Z. D. Chen, "ChipWhisperer: An open-source platform for hardware embedded security research," *Cryptology ePrint Archive*, Paper 2014/204, 2014.
- [28] B. Shakya, T. He, H. Salmani, D. Forte, S. Bhunia, and M. Tehranipoor, "Benchmarking of hardware trojans and maliciously affected circuits," *Journal of Hardware and Systems Security*, vol. 1, pp. 85–102, 2017.
- [29] S. R. Zantout, *Hardware Trojan Detection in FPGA through Side-Channel Power Analysis and Machine Learning*. University of California, Irvine, 2018.
- [30] M. Zhao and G. E. Suh, "FPGA-Based Remote Power Side-Channel Attacks," in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 229–244.
- [31] T. Sugawara, K. Sakiyama, S. Nashimoto, D. Suzuki, and T. Nagatsuka, "Oscillator without a Combinatorial Loop and its Threat to FPGA in Data Center," *Electronics Letters*, vol. 55, 05 2019.
- [32] T.-T. Wong and P.-Y. Yeh, "Reliable Accuracy Estimates from k-Fold Cross Validation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 32, no. 8, pp. 1586–1594, 2020.
- [33] S. Yang, P. Chakraborty, and S. Bhunia, "Side-channel Analysis for Hardware Trojan Detection using Machine Learning," in *2021 IEEE International Test Conference India (ITC India)*, 2021.
- [34] T. Hu, L. Wu, X. Zhang, and Z. Liao, "Hardware Trojan Detection Combines with Machine Learning: an Isolation Forest-based Detection Method," in *2020 IEEE 14th International Conference on Big Data Science and Engineering (BigDataSE)*, 2020, pp. 96–103.